

Data Structures and Algorithms

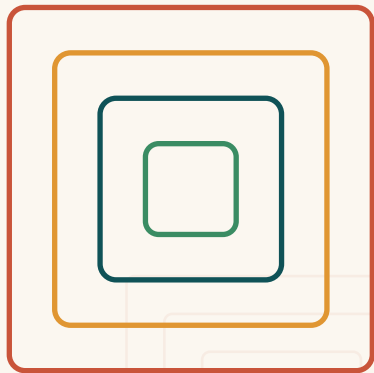
Recursion

A gentle, step-by-step introduction

Md. Owaliur Rahman

Presidency University

September 28, 2026



| Today's Goal

- We are **not** trying to cover a lot of recursion.
- We want just **one** thing: a clear picture in your head of *how recursion works*.
- By the end, you should be able to **trace** a small recursive program by hand.

Remember these three words

Base case • **Smaller problem** • **Coming back**

PART 1

Why Recursion?

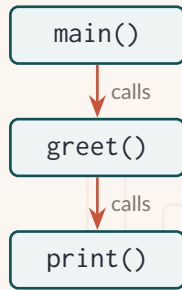


A Quick Reminder: What is a Function?

- A function is a **small machine**: input in, a job done.
- A function can **call another function**.

```
void print(void) {  
    printf("Hi!\n");  
}  
void greet(void) {  
    print();    // -> print  
}  
int main(void) {  
    greet();    // -> greet  
    return 0;  
}
```

The call flow





A function can call another function.

Can a function call *itself*?



| The Idea Behind Recursion

An everyday analogy

You are standing in a long line and want to know your position.

Instead of counting everyone, you ask the person in front:

"What is your position?"

Then your answer is simply **their answer** + 1.

- Each person solves a **smaller** version of the same question.
- The first person in line just says "**1**" and stops.

Definition (keep it simple)

Recursion

Solving a problem by solving a
smaller version of the same problem.

That's the whole idea. Everything else is just detail.

PART 2

Our First Recursive Function

2



| A Tiny Example

```
void hello(int n)
{
    if (n == 0)        // base case: stop here
        return;

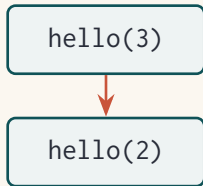
    printf("Hello\n");
    hello(n - 1);      // call itself, smaller
}
```

What happens when we call hello(3)?

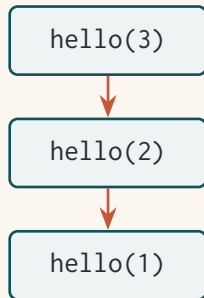
| Tracing hello(3) – going down

hello(3)

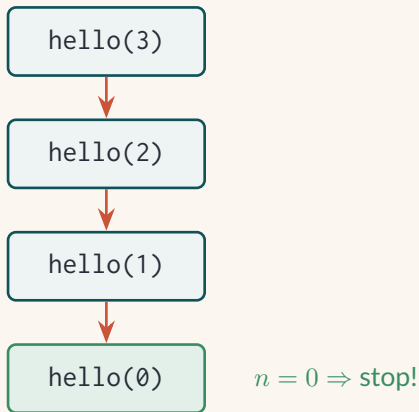
Tracing hello(3) – going down



Tracing hello(3) – going down



Tracing hello(3) – going down



Each call prints “Hello”, then asks a **smaller** call to continue.

The Four Key Ideas

- **Base case:** `if (n == 0) return;`
the place where recursion *stops*.
- **Recursive call:** `hello(n - 1);`
the function calling itself.
- **Why $n - 1$?**
each call gets a *smaller* number.
- **Why it stops:**
 n keeps shrinking: $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$.

What If We Forget the Base Case?

```
void hello(int n)
{
    printf("Hello\n");
    hello(n - 1);    // no base case!
}
```

No stopping point

n goes $3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow -1 \rightarrow -2 \rightarrow \dots$ forever.

The program **never stops** and eventually crashes.

Lesson: every recursion *must* have a base case.

PART 3

The Call Stack

3





**When `hello(3)` calls `hello(2)`, does
`hello(3)` disappear?**



Our Example

```
void fun(int n)
{
    if (n == 0)
        return;

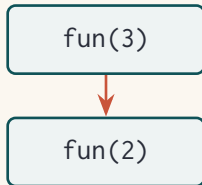
    printf("%d\n", n);
    fun(n - 1);
}
```

We call fun(3). Let's watch memory.

Calls Pile Up – going down

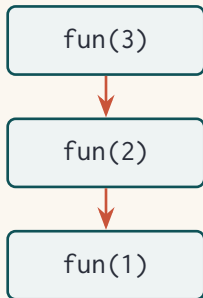
fun(3)

Calls Pile Up – going down



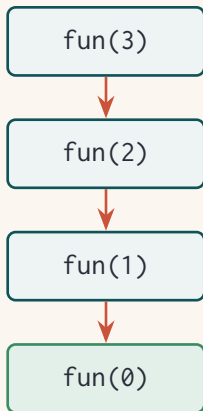
fun(3) is **still there** – it is *waiting* for fun(2) to finish.

Calls Pile Up – going down



fun(3) is **still there** – it is *waiting* for fun(2) to finish.

Calls Pile Up – going down



`fun(3)` is **still there** – it is *waiting* for `fun(2)` to finish.

| Then They Return – coming back up

fun(3)

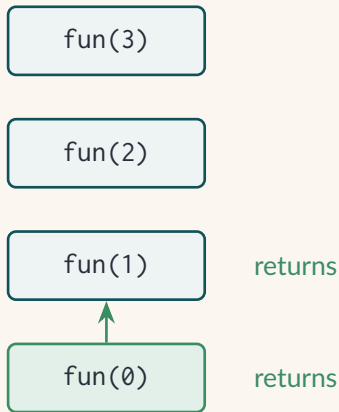
fun(2)

fun(1)

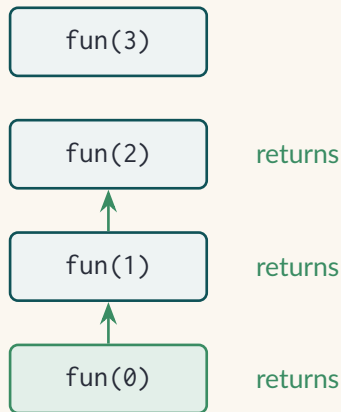
fun(0)

returns

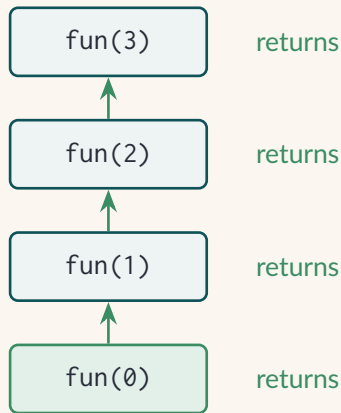
| Then They Return – coming back up



| Then They Return – coming back up



| Then They Return – coming back up



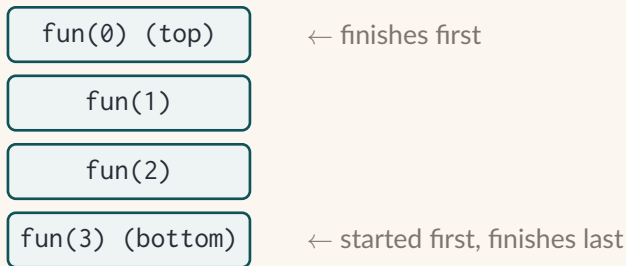
The **last** call to start is the **first** to finish.

| The Big Rule of the Call Stack

Each call waits

A recursive call does **not** replace the previous one.

Every call **waits** for the next call to finish
before it can finish itself.



PART 4

Before vs. After the Recursive Call

4



Two Tiny Programs – spot the difference

Version A

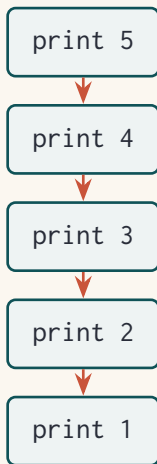
```
printf("%d ", n);  
fun(n - 1);
```

Version B

```
fun(n - 1);  
printf("%d ", n);
```

We call `fun(5)`. Will the output be the same?

Version A: print *then* recurse

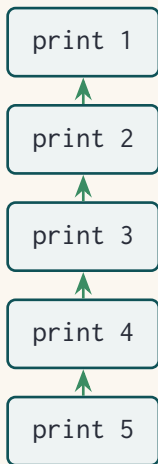


Printing happens **on the way down**.

Output

5 4 3 2 1

Version B: recurse *then* print



Printing happens **on the way back up**.

Output

1 2 3 4 5

| The Takeaway

Where your code sits matters

- Code **before** the recursive call
⇒ runs while **going down**.
- Code **after** the recursive call
⇒ runs while **coming back**.

Same calls, different *timing* of the work.

PART 5

First Real Problem: Print 1 to N

5



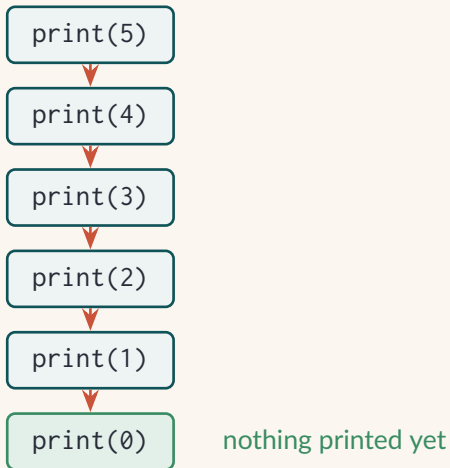
Print 1 to N

```
void print(int n)
{
    if (n == 0)
        return;

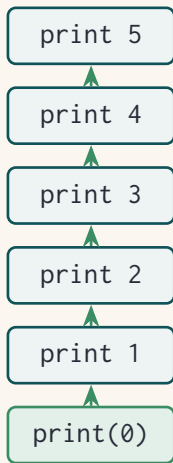
    print(n - 1);    // go smaller first
    printf("%d ", n); // print on the way back
}
```

Let's trace print(5).

Going Down: keep shrinking



Coming Back: now we print



Output

1 2 3 4 5

The smallest call prints first, so the numbers come out in increasing order.

| One Small Change $\Rightarrow$ Reverse Order

```
void print(int n)
{
    if (n == 0)
        return;

    printf("%d ", n); // print FIRST
    print(n - 1);     // then go smaller
}
```

Output

5 4 3 2 1

Print *before* the call $\Rightarrow$ work done on the way **down** $\Rightarrow$ decreasing order.

PART 6

Factorial



Start With the Maths

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Look carefully:

$$5! = 5 \times \underbrace{(4 \times 3 \times 2 \times 1)}_{= 4!}$$

So $5! = 5 \times 4!$

Find the Pattern

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2!$$

$\vdots$

General rule and stopping point

$$n! = n \times (n - 1)! \quad \text{and} \quad 0! = 1$$

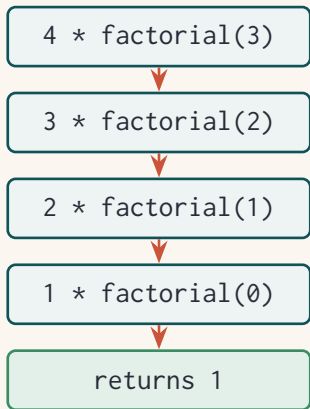
Factorial in C

```
int factorial(int n)
{
    if (n == 0)
        return 1;           // base case

    return n * factorial(n - 1);
}
```

- **Base case:** $0! = 1$.
- **Smaller problem:** `factorial(n - 1)`.

Tracing factorial(4) – going down



Every multiplication is **waiting** for the call on its right.

| Now Evaluate – coming back up

`factorial(0)` = 1
`factorial(1)` = $1 \times 1 = 1$
`factorial(2)` = $2 \times 1 = 2$
`factorial(3)` = $3 \times 2 = 6$
`factorial(4)` = $4 \times 6 = \mathbf{24}$

Read from top to bottom: results build up on the way back.

PART 7

Recursion vs. Iteration



Same Job, Two Styles

With a loop

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <=
        n; i++)
        result *= i;
    return result;
}
```

With recursion

```
int factorial(int n)
{
    if (n == 0)
        return 1;
    return n * factorial(
        n - 1);
}
```

Both give the same answer.

| How to Think About It

- Recursion is just **another way** to express repetition.
- It is **not automatically better** than a loop.
- For some problems, recursion is **easier to write and read**.
- Recursion uses **stack memory** (every call is stored).
- Too many calls with no base case $\Rightarrow$ **stack overflow** (crash).

PART 8

Common Mistakes

8



Mistake 1 – No Base Case

```
void fun(int n)
{
    printf("%d", n);
    fun(n - 1);    // never stops
}
```

What goes wrong

There is no if that says “stop”. The calls pile up forever until memory runs out ⇒ **stack overflow**.

Mistake 2 – Moving Away From the Base Case

```
void fun(int n)
{
    if (n == 0)
        return;

    fun(n + 1);    // n grows!
}
```

Ask yourself

Starting at $n = 3$: $3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow \dots$

Will this ever reach 0? (No.) The base case is never hit.

Mistake 3 – Wrong Base-Case Value

```
int factorial(int n)
{
    if (n == 0)
        return 0;    // should be 1!

    return n * factorial(n - 1);
}
```

Why every answer becomes 0

Everything is multiplied by that final 0:

$$n \times \cdots \times 1 \times 0 = 0.$$

PART 10

Guided Practice

10



| How We'll Practise

For each problem:

1. Look at the problem.
2. Find the **base case**.
3. Find the **smaller problem**.
4. *Then* we look at the code.

Try each one in your head before the answer appears.

| Exercise 1 – Count the Digits

Problem: how many digits are in a number? (e.g. 5324 $\rightarrow$ 4)

Exercise 1 – Count the Digits

Problem: how many digits are in a number? (e.g. $5324 \rightarrow 4$)

- Base case: $n == 0 \Rightarrow \text{return } 0$.
- Smaller problem: drop one digit with $n / 10$.

Exercise 1 – Count the Digits

Problem: how many digits are in a number? (e.g. 5324 $\rightarrow$ 4)

- Base case: $n == 0 \Rightarrow$ return 0.
- Smaller problem: drop one digit with $n / 10$.

```
int digits(int n)
{
    if (n == 0) return 0;
    return 1 + digits(n / 10);
}
```

| Exercise 2 – Fibonacci

Problem: the sequence 0, 1, 1, 2, 3, 5, 8, ... where each number is the sum of the two before it.

Exercise 2 – Fibonacci

Problem: the sequence 0, 1, 1, 2, 3, 5, 8, ... where each number is the sum of the two before it.

- Base cases: $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$.
- Smaller problems: $\text{fib}(n-1)$ and $\text{fib}(n-2)$.

Exercise 2 – Fibonacci

Problem: the sequence 0, 1, 1, 2, 3, 5, 8, ... where each number is the sum of the two before it.

- Base cases: $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$.
- Smaller problems: $\text{fib}(n-1)$ and $\text{fib}(n-2)$.

```
int fib(int n)
{
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}
```

This one calls itself **twice** – our first branching recursion.

Exercise 3 – Min Cost Climbing Stairs

Problem: each stair `cost[i]` has a price. From a stair you may climb **1 or 2** steps; you may start at stair 0 or 1. Find the **cheapest** way to get *just past the last stair*.

Exercise 3 – Min Cost Climbing Stairs

Problem: each stair $\text{cost}[i]$ has a price. From a stair you may climb **1 or 2** steps; you may start at stair 0 or 1. Find the **cheapest** way to get *just past the last stair*.

- Base case: $i \geq n \Rightarrow \text{return } 0$ (already at the top).
- Two choices from stair i : step to $i+1$ or $i+2$.
- Take the cheaper: $\text{cost}[i] + \min(f(i+1), f(i+2))$.

Exercise 3 – The Recursive Solution

```
int minCost(int i, int cost[], int n)
{
    if (i >= n) return 0;           // reached the top
    int a = minCost(i + 1, cost, n);
    int b = minCost(i + 2, cost, n);
    return cost[i] + (a < b ? a : b); // smaller
}
```

Exercise 3 – The Recursive Solution

```
int minCost(int i, int cost[], int n)
{
    if (i >= n) return 0;           // reached the top
    int a = minCost(i + 1, cost, n);
    int b = minCost(i + 2, cost, n);
    return cost[i] + (a < b ? a : b); // smaller
}
```

The answer

Start at stair 0 or 1 – whichever ends up cheaper:

```
min( minCost(0, cost, n), minCost(1, cost, n) )
```

PART 11

Quick Concept Check



Can You Answer These?

- What is a **base case**?
- Why must the input move **toward** the base case?
- What happens if there is **no** base case?
- What is stored in the **call stack**?
- Why does moving `printf` before/after the call change the output?
- What is the difference between **recursion** and **iteration**?

Discuss with the person next to you.

| The Big Idea

Every recursive solution needs

1. A stopping point
2. A smaller version of the same problem
3. A recursive call that eventually reaches the stopping point

That's recursion. You've got this.

Practice on Your Own

Use the checklist every time: **base case** → **smaller problem** → **recursive call**.

EASY

- Print $1 \dots N / N \dots 1$
- Sum $1 + 2 + \dots + N$
- Factorial $n!$
- Power a^n
- Count / sum of digits

MEDIUM

- Reverse a string
- Check a palindrome
- GCD (Euclid)
- Sum / max of an array
- Binary search

HARD

- Tower of Hanoi
- Ways to climb stairs
- String permutations
- All subsets of a set
- Merge sort



Thank You

Questions?

